

Industrial Oriented Mini Project (CM653PC)

on

AI Smart Fitness Trainer using Computer Vision

Submitted

In partial fulfillment of the requirements for the award of the degree of

Bachelor of Technology

in

Computer Science Engineering
(Artificial Intelligence and Machine Learning)

by

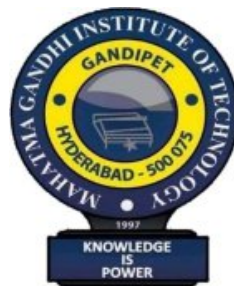
V. PHANIDEEP REDDY (23261A6661)

&

Y. VENKATA SURYA DHANUSH (23261A6662)

Under the guidance of

K. DIVIJA



(Assistant Professor)

Department of Emerging Technologies

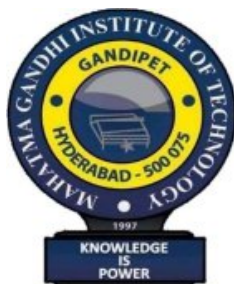
Mahatma Gandhi Institute of Technology (Autonomous)

(Affiliated to Jawaharlal Nehru Technological University Hyderabad)

Kokapet, Gandipet, Hyderabad. Telangana - 500075

2025-2026

MAHATMA GANDHI INSTITUTE OF TECHNOLOGY (AUTONOMOUS)
(Affiliated To Jawaharlal Nehru Technological University Hyderabad)
GANDIPET, HYDERABAD- 500075, Telangana



CERTIFICATE

This is to certify that the project entitled "AI Smart Fitness Trainer using Computer Vision" is being submitted by V. PHANIDEEP REDDY (23261A6661), Y. VENKATA SURYA DHANUSH (23261A6662) in partial fulfillment of the requirements for the Industrial Oriented Mini Project (CM653PC) in COMPUTER SCIENCE AND ENGINEERING (AI & ML) and is a record of bonafide work carried out by them under our guidance and supervision.

The results of the investigations enclosed in this report have been verified and found satisfactory.

PROJECT GUIDE	HEAD OF THE DEPARTMENT
K. DIVIJA Assistant Professor, Dept of ET	M. RAMA BAI Professor, Dept of ET

EXTERNAL EXAMINER

DECLARATION

This is to certify that the work reported in this project titled "AI Smart Fitness Trainer using Computer Vision" is a record of work done by us in the Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Mahatma Gandhi Institute of Technology, Hyderabad.

No part of the work is copied from books/journals/internet and wherever the portion is taken, the same has been duly referred to in the text. The report is based on the work done entirely by us and not copied from any other source.

V. PHANIDEEP REDDY
23261A6661

Y. VENKATA SURYA DHANUSH
23261A6662

ACKNOWLEDGEMENT

The satisfaction that accompanies the successful completion of any task would be incomplete without the mention of people who made it possible because success is the abstract of hardwork and perseverance, but steadfast of all is encouraging guidance. So, we acknowledge all those whose guidance and encouragement served as a beacon light and crowned our efforts with success.

We would like to express our sincere thanks to, Prof G. ChandraMohan Reddy, Principal MGIT, for providing the working facilities in college.

We wish to express our sincere thanks and gratitude to Prof. M. Ramabai, HoD, Department of ET, MGIT, for all the timely support and valuable suggestions during the period of the project.

We are extremely thankful and indebted to our mentors Ms. K Divija, Assistant Professors from the Department of Emerging Technologies for their constant guidance, encouragement and moral support throughout the project.

Finally, we would also like to thank all the faculty and staff of the ET Department who helped us directly or indirectly in completing this project.

V. PHANIDEEP REDDY
23261A6661

Y. VENKATA SURYA DHANUSH
23261A6662

TABLE OF CONTENTS

TOPIC	PAGE NO.
Certificate	I
Declaration	II
Acknowledgement	III
List of Figures	V
List of Tables	VI
Abstract	VII
1. Introduction	1
1.1 Introduction to the Project	1
1.2 Existing System	1
1.3 Problem Definition	2
1.4 Proposed System	3
1.5 Requirements Specification	3
2. Literature Survey	5
3. Design and Methodology	7
4. Flowchart	8
5. Implementation and Results	10
6. Conclusion and Future Scope	16
Bibliography	17
Appendix	18

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
Figure 3.1.1	Flow chart	8
Figure 4.7.1	Home Screen UI – AI Fitness Trainer Dashboard with Workout Controls and Live Coaching Preview	12
Figure 4.7.2	Performance Dashboard – Real-Time Metrics Including Reps, Angle, Accuracy, and Duration	13
Figure 4.7.3	Interactive Exercise Selection Interface – Bicep Curls, Squats, and Pushups Cards	13
Figure 4.7.4	System Workflow Overview – Exercise Selection, Workout Start, and Real-Time Coaching Process	14
Figure 4.7.5	AI Fitness Trainer – Perfect Posture Detection with 100% Accuracy and Skeleton Alignment Visualization	14
Figure 4.7.6	AI Smart Fitness Trainer – Live Bicep Curl Tracking with Rep Count, Angle Detection, and Real-Time Form Correction	15

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
Table 2.1	Literature Survey Summary	6
Table 4.2.1	Key Front-End Components	10
Table 4.3.1	Key Back-End Components	10
Table 4.5.1	Key Features Implemented	11
Table 4.7.1	Result Analysis	12

ABSTRACT

The AI Smart Fitness Trainer is a real-time computer vision based workout coaching system developed to help users perform exercises with better posture, repetition tracking, and immediate feedback. The application uses a webcam feed to observe the user, detects body landmarks using MediaPipe Pose, and evaluates movement patterns for exercises such as bicep curls, squats, and pushups. The processed result is shown as a live video stream with a colored skeleton, repetition counter, angle display, and posture feedback.

The system is implemented as a full-stack application with a React, TypeScript, Vite, and Tailwind CSS frontend and a Python FastAPI backend powered by OpenCV, MediaPipe, NumPy, and Pydantic. The frontend lets the user choose an exercise and explicitly start or stop the workout session, while the backend controls the camera, runs pose estimation, computes joint angles, counts repetitions, scores form accuracy, and streams annotated frames. The interface also includes browser-based voice coaching so that rep counts and corrective instructions can be heard during a session.

By combining lightweight pose tracking, exercise-specific heuristics, and an interactive web dashboard, the project delivers an affordable and user-friendly fitness assistant that can run on common laptop hardware. The system demonstrates how artificial intelligence and computer vision can support safer home workouts, improve user awareness of movement quality, and serve as a strong foundation for future personalized training applications.

1. INTRODUCTION

1.1 Introduction to the Project

The rapid growth of artificial intelligence and computer vision has opened new possibilities for personal health and fitness applications. Real-time pose estimation makes it possible for a computer to observe human movement, extract key body landmarks, and interpret exercise posture without requiring special sensors. This project uses that capability to build an AI Smart Fitness Trainer that provides live workout guidance through a standard webcam and a browser-based dashboard.

Many people exercise at home without access to a personal trainer. As a result, they may perform movements with incorrect posture, incomplete range of motion, or poor body alignment. Such mistakes reduce workout quality and can increase the risk of strain or injury. A system that can observe the user, count repetitions correctly, and explain what needs to be corrected can make self-guided workouts more effective and safer.

The AI Smart Fitness Trainer is implemented as a full-stack web application. The frontend is developed using React with TypeScript, Vite, and Tailwind CSS to create a responsive single-page interface. The backend is developed using Python FastAPI with OpenCV for webcam processing and MediaPipe Pose for landmark detection. This architecture keeps the heavy camera and vision pipeline on the backend while the frontend focuses on user interaction and presentation.

The working flow begins when the user selects an exercise and presses Start Workout. The backend then opens the webcam, processes each frame, detects landmarks, resolves the best body side to track, and calculates exercise-specific joint angles. Based on these angles, the system counts repetitions, checks posture rules, calculates a current accuracy value, and computes a rolling average. The processed frame and live status are returned to the frontend, where the user can monitor progress visually.

A major strength of the project is the combination of multiple coaching signals in a single interface. The user receives a live annotated stream, green or red skeleton feedback, repetition counts, angle values, accuracy scores, duration tracking, and spoken coaching prompts through the browser's speech synthesis API. The camera also remains under explicit user control, since the workout only starts after exercise selection and a deliberate start action.

Overall, the project demonstrates how a relatively lightweight combination of pose landmark detection, rule-based exercise analysis, and modern web technologies can be used to create a practical fitness assistant. It is suitable for demonstrations, student projects, and as a foundation for more advanced real-time workout coaching systems.

1.2 Existing System

Existing digital fitness systems usually fall into three categories: video-based tutorial

platforms, workout timer applications, and wearable sensor based monitoring systems. Tutorial platforms can demonstrate exercises, but they do not understand the user's posture. Timer applications track duration and sets, but they cannot verify movement quality. Wearable devices capture useful motion data, but they often require dedicated hardware and may not provide intuitive visual feedback.

Some recent systems use pose estimation in the browser or on mobile devices to track fitness activity. Although these tools show the promise of camera-based coaching, many of them are limited to simple landmark visualization or only one exercise type. They may also struggle with noisy webcam input, unclear repetition boundaries, or a lack of exercise-specific feedback that explains how the user should improve.

Another limitation of many existing solutions is the lack of coordinated system design between the frontend and backend. When the entire pipeline is forced into the browser, performance can become inconsistent across systems. When the backend exists but does not expose live metrics cleanly, the user experience becomes disconnected. In addition, several available tools do not combine streaming video, repetition counting, posture validation, and spoken coaching in one interface.

Therefore, while current systems provide partial support for digital exercise tracking, they often fail to offer a complete, real-time, low-cost solution that works with a normal webcam and gives users meaningful form feedback during each repetition.

1.3 Problem Definition

People performing home workouts often need immediate and reliable feedback about how well they are moving. Without supervision, users may fail to reach full exercise range, keep poor joint alignment, or lose posture consistency across repetitions. These issues are difficult to identify from memory alone, especially during dynamic movements such as curls, squats, and pushups.

There is a need for a system that can capture live webcam input, detect human pose landmarks in real time, analyze movement angles, and convert those measurements into practical workout guidance. The system should count repetitions only when the required movement thresholds are met, distinguish correct and incorrect form, and provide feedback in both visual and spoken form. It should also be easy to run using commonly available development tools and standard laptop hardware.

Hence, the problem addressed in this project is the design and implementation of a real-time smart fitness trainer that uses computer vision to monitor exercise form, count repetitions, stream annotated video, and provide intuitive coaching feedback through a web interface.

OBJECTIVES

The specific objectives of the project are:

1. To detect human body landmarks from live webcam frames using MediaPipe

Pose.

2. To support real-time tracking for bicep curls, squats, and pushups.
3. To count repetitions using exercise-specific angle transitions and movement stages.
4. To validate posture and range of motion and generate corrective feedback messages.
5. To show a processed live stream with green or red skeleton visualization.
6. To present workout metrics such as reps, angle, accuracy, duration, and landmarks.
7. To provide voice coaching and user-controlled session start and stop behavior.

1.4 Proposed System

The proposed system is a full-stack AI Smart Fitness Trainer that uses a standard webcam to observe a user during exercise and delivers live coaching through a web dashboard. The system focuses on three supported movements: bicep curls, squats, and pushups. For every frame, the posture engine evaluates joint geometry and movement quality and converts the analysis into clear user-facing feedback.

The frontend is developed using React, TypeScript, Vite, and Tailwind CSS. It provides exercise cards, session controls, a live studio view, a metrics dashboard, and a workout overlay. The user selects an exercise, then presses Start Workout to request the backend to begin processing. The frontend also contains a voice coaching hook that uses the browser speech synthesis API to speak repetition counts and form warnings.

The backend is developed using FastAPI and manages the full vision pipeline. It opens the webcam, reads frames with OpenCV, and runs MediaPipe Pose to detect landmarks. A dedicated pose processor computes landmark payloads, draws a live HUD, chooses the better tracked body side, and invokes exercise logic that calculates joint angles, identifies movement stage transitions, increments the counter, and produces exercise-specific feedback.

The system is designed to overcome the limitations of generic workout applications by combining real-time pose tracking, movement-specific heuristics, annotated video streaming, and a synchronized frontend-backend workflow. The session manager keeps track of duration, camera backend, accuracy history, and the latest video frame so that the user receives both a live stream and regularly refreshed JSON status updates.

Overall, the proposed solution offers a practical and extensible approach for smart fitness coaching. It is cost-effective, does not require extra wearable hardware, and is suitable for local demonstrations, prototype deployments, and future extensions such as more exercises, user history, and personalized training analytics.

1.5 Requirements Specification

1.5.1 Software Requirements

- Operating System: Windows 10 or later / Linux / macOS
- Programming Languages: Python 3.10 to 3.12, TypeScript, HTML, CSS
- Frontend Framework: React 18 with Vite
- Backend Framework: FastAPI with Uvicorn
- Computer Vision Libraries: MediaPipe Pose, OpenCV, NumPy
- State and Validation Libraries: Pydantic, browser Fetch API
- UI and Icon Libraries: Tailwind CSS, Lucide React
- Speech Module: Web Speech API (SpeechSynthesis)
- API Communication: JSON over HTTP between frontend and backend
- Build Tools: Node.js 18+, npm, Vite, TypeScript compiler
- Development Environment: Visual Studio Code / PyCharm / any modern IDE
- Version Control: Git, GitHub
- Testing and Debugging: Browser DevTools, terminal logs, API testing tools
- Deployment: Localhost demo setup / LAN access / optional cloud extension

1.5.2 Hardware Requirements

The hardware listed below is sufficient for local execution of the smart fitness trainer with real-time webcam analysis and web dashboard rendering.

- Processor: Intel i5 or higher / AMD equivalent
- RAM: Minimum 8 GB (16 GB recommended for smoother multitasking)
- Storage: Minimum 5 GB free space for dependencies and project files
- GPU (Optional): Dedicated GPU is helpful but not required
- Webcam: Built-in or external webcam with stable video capture
- Display: Standard laptop or desktop monitor
- Internet Connection: Required for dependency installation and optional updates
- Input Devices: Keyboard and mouse or touchpad
- Audio Output: Speakers or headphones for voice coaching feedback

2. LITERATURE SURVEY

Recent work in human pose estimation has made real-time exercise analysis more practical and accessible. Early deep learning systems such as OpenPose established the effectiveness of landmark-based body tracking in images and videos. Later frameworks such as MediaPipe simplified deployment by providing reusable perception pipelines and optimized pose models that can run efficiently on common devices.

For fitness applications, pose landmarks are especially valuable because they allow exercise logic to be expressed in terms of joint geometry rather than raw pixels. Official pose classification examples from Google show that repetition counting can be built from normalized landmarks, threshold crossings, and rule-based analysis. This approach is well suited to exercises such as pushups and squats where stage transitions can be identified through changing joint angles.

Beyond vision models, the design of a complete workout assistant also depends on supporting technologies. OpenCV provides dependable camera access and image processing primitives, while FastAPI offers a lightweight API layer for session control and streaming endpoints. Pydantic improves reliability through typed request models, and modern frontend tools such as React, Vite, and Tailwind CSS simplify the creation of responsive, data-driven interfaces.

Accessibility is another important aspect of workout guidance. Browser speech synthesis enables the system to speak rep counts and posture corrections so that the user does not need to watch the screen continuously. This improves usability during exercise, particularly when the user is focused on movement rather than on reading status panels.

The literature and official technical references together show that a strong smart fitness system can be built by combining real-time landmark detection, angle-based movement heuristics, efficient backend communication, and an interactive frontend. The proposed project follows this integrated approach and adapts it to a practical three-exercise fitness coaching application.

Table 2.1: Literature Survey Summary

S.No	Author(s)	Year	Title	Methodology/ Technology Used	Key Findings
1	Cao et al.	2017	Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields	PAF-based pose estimation	Established a strong real-time landmark detection baseline.
2	Lugaresi et al.	2019	MediaPipe: A Framework for Building Perception Pipelines	Graph-based perception pipeline	Enabled modular and cross-platform vision applications.
3	Grishchenko et al.	2022	BlazePose GHUM Holistic	Lightweight landmark model	Delivered fast full-body landmarks suited for fitness tracking.
4	Google for Developers	2024	Pose Detection	33-landmark pose API	Provided practical real-time body tracking for apps.
5	Google for Developers	2024	Pose Classification Options	k-NN and angle heuristics	Showed repetition counting and pose interpretation strategies.
6	Bradski	2000	The OpenCV Library	Computer vision library	Supplied core image and video processing primitives.
7	OpenCV Team	2024	VideoCapture Class Reference	Video I/O API	Supported flexible webcam access and backend selection.
8	Ramirez	2024	FastAPI Documentation	Python web framework	Simplified REST endpoints and streaming responses.
9	Pydantic Team	2024	Models and BaseModel	Typed data validation	Improved request validation and API reliability.
10	React Team	2024	React Documentation	Component-based UI	Enabled reactive rendering for the workout dashboard.
11	Vite Team	2024	Getting Started	Frontend build tooling	Improved TypeScript React development speed.
12	Tailwind Labs	2024	Styling with Utility Classes	Utility-first CSS	Accelerated responsive UI design and consistency.
13	MDN Web Docs	2025	Web Speech API	Browser speech synthesis	Added accessibility through text-to-speech support.
14	NumPy Developers	2024	Array Objects and ndarray	Numerical array processing	Supported efficient angle and coordinate calculations.
15	MDN Web Docs	2025	SpeechSynthesis	Browser TTS interface	Enabled direct spoken feedback in workout sessions.

3. DESIGN AND METHODOLOGY

The AI Smart Fitness Trainer is designed using a modular client-server architecture that separates the user interface from the pose analysis engine. The React frontend manages exercise selection, state presentation, and voice feedback, while the FastAPI backend manages the webcam, pose estimation, and exercise logic. This division keeps the overall system organized and makes the project easier to test, extend, and demonstrate.

On the frontend, the application exposes a workflow that mirrors how a user would actually begin a workout session. Exercise cards allow the user to choose a target movement, and the camera remains off until Start Workout is pressed. Once a session starts, the live stream area, workout overlay, and insight cards continuously present the latest counter, angle, duration, and posture metrics.

On the backend, a workout session manager is responsible for session lifecycle, frame storage, duration tracking, and coordination between the API and a background camera thread. The camera loop attempts multiple OpenCV backends to improve compatibility on Windows systems. Every captured frame is passed to the pose processor, which mirrors the frame, runs MediaPipe Pose, and transforms the landmarks into structured workout status data.

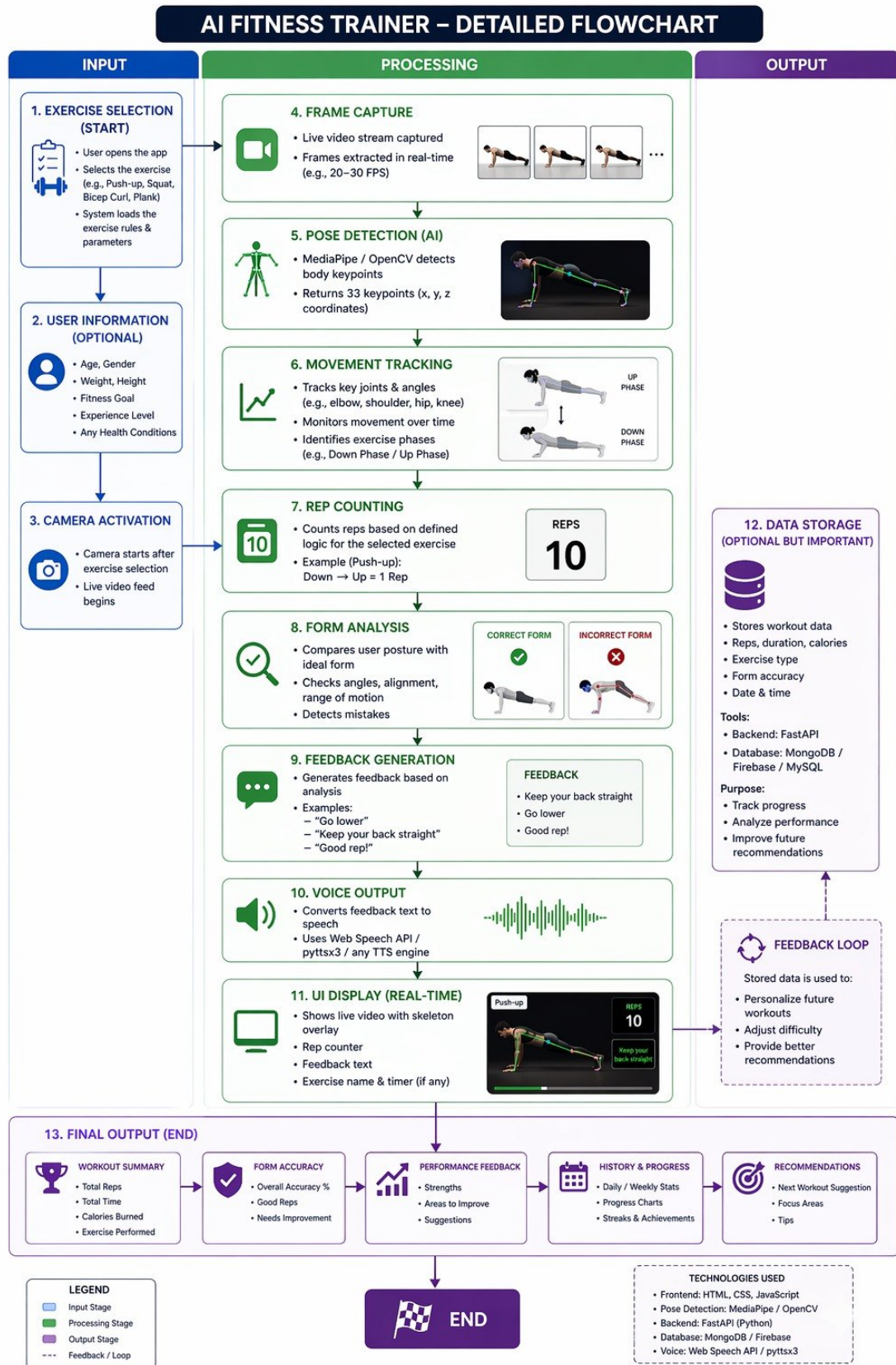
The exercise logic module uses joint angle calculations, visibility thresholds, and movement-state transitions to evaluate each repetition. Bicep curls focus on shoulder-elbow-wrist geometry and elbow stability, squats monitor hip-knee-ankle depth with torso posture, and pushups evaluate shoulder-elbow-wrist motion together with shoulder-hip-ankle alignment. This rule-based approach makes the decision path transparent and easy to tune.

A rolling accuracy history is maintained so that the user sees both momentary form quality and a more stable average over recent frames. The processed frame is encoded as JPEG and streamed back to the frontend using an MJPEG endpoint, while a separate status endpoint returns the numeric metrics and feedback message. This dual-channel design ensures that visual and textual feedback remain synchronized.

The overall methodology favors practical deployment over heavyweight training. The system relies on a pre-trained landmark detector and carefully designed heuristics instead of building a custom dataset and model from scratch. This makes the project lightweight, reproducible, and suitable for academic demonstration using local development tools.

3.1 FLOWCHART

Fig. no. 3.1.1: Flow chart of AI Smart Fitness Trainer



3.1.1 Flowchart Explanation

The flowchart represents the working process of the AI Smart Fitness Trainer. The user first selects an exercise and starts the session from the frontend. The backend then opens the webcam, captures frames, and runs pose estimation. Landmark data is analyzed using exercise-specific logic, after which repetition count, stage, accuracy, and feedback are updated. The processed frame and live metrics are shown in the interface until the user stops the workout.

3.1.2 System Overview

The system is a web-based application developed using React, TypeScript, and Tailwind CSS on the frontend and Python FastAPI, OpenCV, and MediaPipe on the backend. It captures live webcam input, performs pose-aware exercise analysis, and returns visual and spoken coaching in near real time.

3.1.3 Dataset Design and Preparation

The project does not require a custom training dataset because it relies on the pre-trained MediaPipe Pose model for landmark detection. Instead of dataset collection, the project uses calibrated angle thresholds and exercise heuristics derived from common movement patterns for bicep curls, squats, and pushups.

3.1.4 Webcam and Session Control Module

This module is responsible for safe camera startup, session state management, frame capture, and stream termination. It ensures that the webcam is opened only after the user explicitly starts a workout and that resources are released cleanly when the session ends.

3.1.5 Data Processing and Classification

In this module, every webcam frame is mirrored, converted to RGB, and processed using MediaPipe Pose. The detected landmarks are converted into joint angles and exercise states. The system then counts repetitions, evaluates posture, computes accuracy, and creates a serialized landmark payload that can be consumed by the frontend.

3.1.6 User Interface Integration

The user interface is developed using React and presents the session through exercise cards, a hero control area, a live studio view, and analytics panels. The frontend communicates with the backend through REST calls for session control and uses a live MJPEG stream for the processed camera feed. Browser speech synthesis is integrated to deliver rep counts and correction messages, improving interactivity during exercise.

4. IMPLEMENTATION AND RESULTS

4.1 Overview

The implementation of the AI Smart Fitness Trainer integrates a real-time pose estimation backend with a modern web interface designed for live coaching. The system follows a client-server architecture in which the frontend manages user interaction and the FastAPI backend controls the camera, pose tracking, repetition logic, and streaming response.

The core functionality allows users to:

- Select an exercise from the dashboard before the camera starts.
- Start and stop a controlled webcam workout session from the frontend.
- View a processed live video stream with exercise overlay and HUD information.
- Receive spoken rep counts and corrective coaching messages during movement.
- Track bicep curls, squats, and pushups with live metrics and posture scoring.

4.2 Front-End Components

The frontend is developed using React, TypeScript, Vite, and Tailwind CSS, providing a responsive and presentation-ready single-page interface.

Table 4.2.1: Key Front-End Components

Component	Description
Exercise Selection Cards	Let the user choose bicep curls, squats, or pushups
Hero Session Controls	Start and stop workout sessions after exercise selection
Live Studio Screen	Displays processed MJPEG stream or an informative empty state
Workout Overlay	Shows reps, angle, accuracy, duration, and live feedback on the stream
Voice Coach Hook	Speaks repetition counts and correction cues using browser speech synthesis

4.3 Back-End Components

The backend is developed using Python FastAPI and handles the pose estimation pipeline, camera control, session state, and streaming endpoints.

Table 4.3.1: Key Back-End Components

Component	Functionality
FastAPI Server	Handles health, start, stop, status, and video feed endpoints
WorkoutSessionManager	Manages camera thread, shared session state, and duration
PoseProcessor	Runs MediaPipe Pose, draws HUD overlays, and tracks rolling accuracy
Exercise Logic	Calculates angles, stages, repetitions, and form scores per exercise

Utility Module	Encodes frames, selects camera backends, and serializes landmarks
----------------	---

4.4 System Integration

The system integrates frontend and backend using REST APIs and MJPEG streaming. The workflow includes:

1. Selecting an exercise and sending a start request from the frontend.
2. Opening the webcam and capturing frames on the backend.
3. Running MediaPipe pose estimation and exercise-specific rule evaluation.
4. Updating the live status payload and encoded video stream.
5. Rendering metrics and speaking coaching cues in the browser.

This ensures smooth communication and near real-time interaction throughout a session.

4.5 Key Features Implemented

Table 4.5.1: Key Features Implemented

Feature	Implementation Highlights
Real-Time Pose Tracking	MediaPipe Pose with live skeleton overlay on webcam frames
Exercise-Specific Rep Counting	Separate logic for curls, squats, and pushups
Form Validation	Angle-based posture checks and corrective feedback messages
Voice Coaching	Speech synthesis announces reps and form corrections
Live Metrics Dashboard	Shows counter, angle, accuracy, duration, and landmark count
Controlled Camera Lifecycle	Webcam starts only after exercise selection and start action

4.6 Sample Results

Here is a sample sequence of system operation:

- User selects the "Bicep Curls" exercise on the dashboard
- Frontend sends a session start request to the backend
- MediaPipe tracks shoulder, elbow, and wrist landmarks in real time

Example Output:

Input: Live webcam frame of a user performing a curl

Status: Counter = 1, Stage = "up", Angle = 42.7 degrees

Feedback: "Nice squeeze at the top."

Overlay: Skeleton becomes green when the posture is correct

During squat or pushup sessions:

- The system continues updating the MJPEG stream and JSON status payload

- The user receives live corrections, rep counts, and rolling accuracy feedback

4.7 Result Analysis

Table 4.7.1: Result Analysis

Metric	Observation
Accuracy	Form score reflects alignment, depth, and movement range in real time.
Response Time	Local MJPEG streaming and status polling support live feedback.
Usability	Single-page dashboard keeps the workout flow simple and clear.
Accessibility	Voice prompts reduce the need to constantly watch the screen.
Robustness	Multiple camera backend attempts improve webcam compatibility on Windows.
Performance	Rolling average accuracy and lightweight rules keep the session smooth.

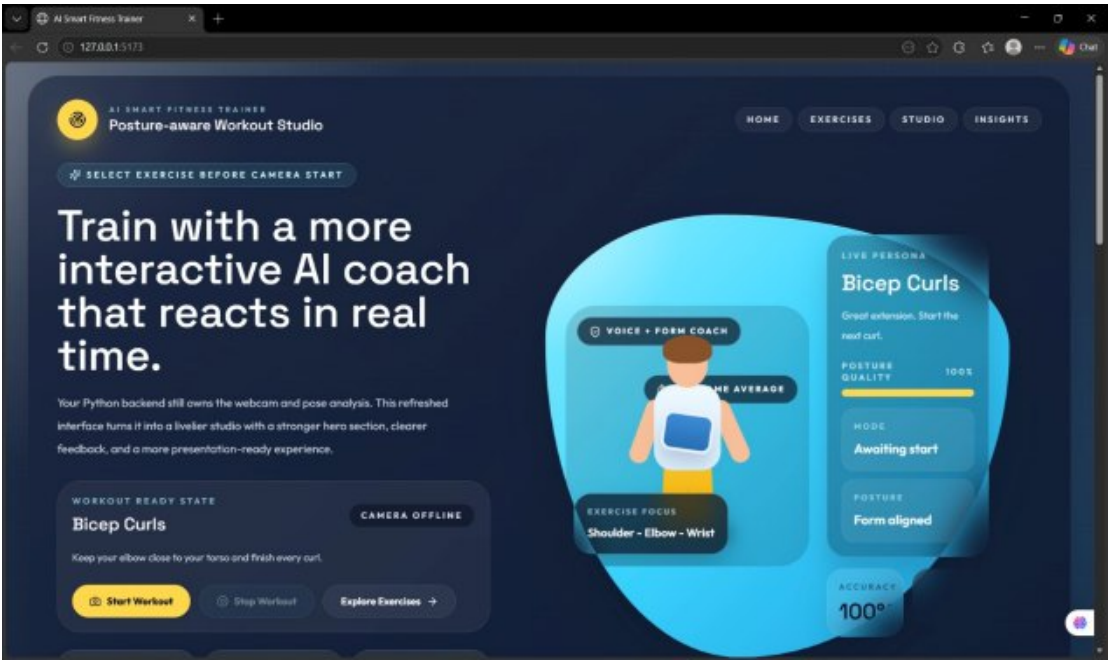


Fig. no. 4.7.1: Home Screen UI – AI Fitness Trainer Dashboard with Workout Controls and Live Coaching Preview

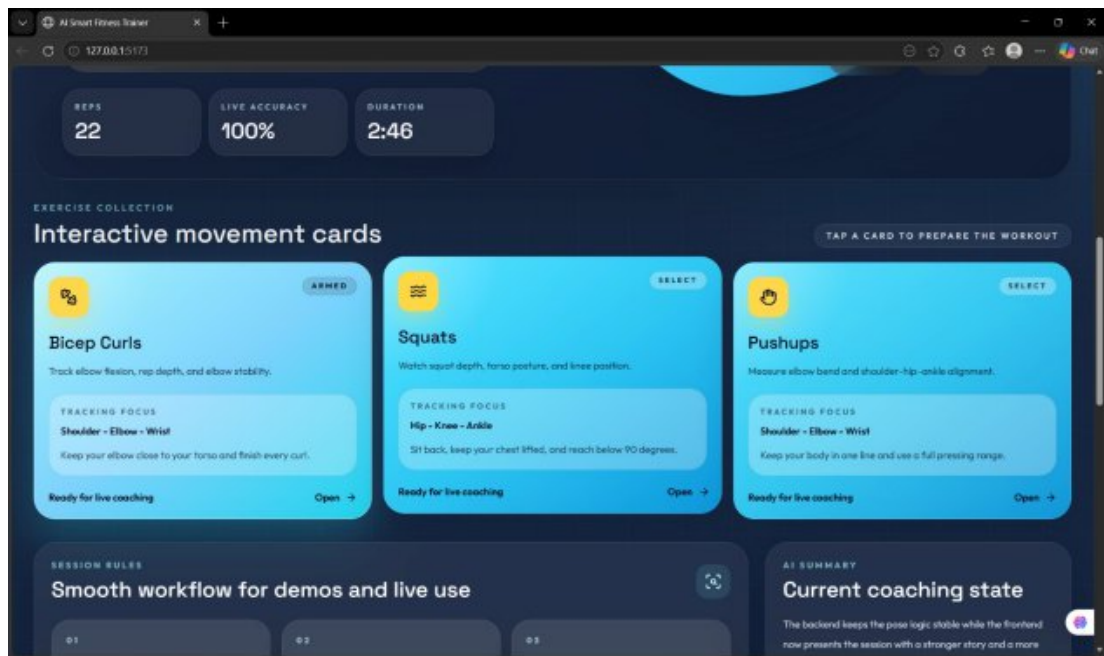


Fig. no. 4.7.2: Performance Dashboard – Real-Time Metrics Including Reps, Angle, Accuracy, and Duration

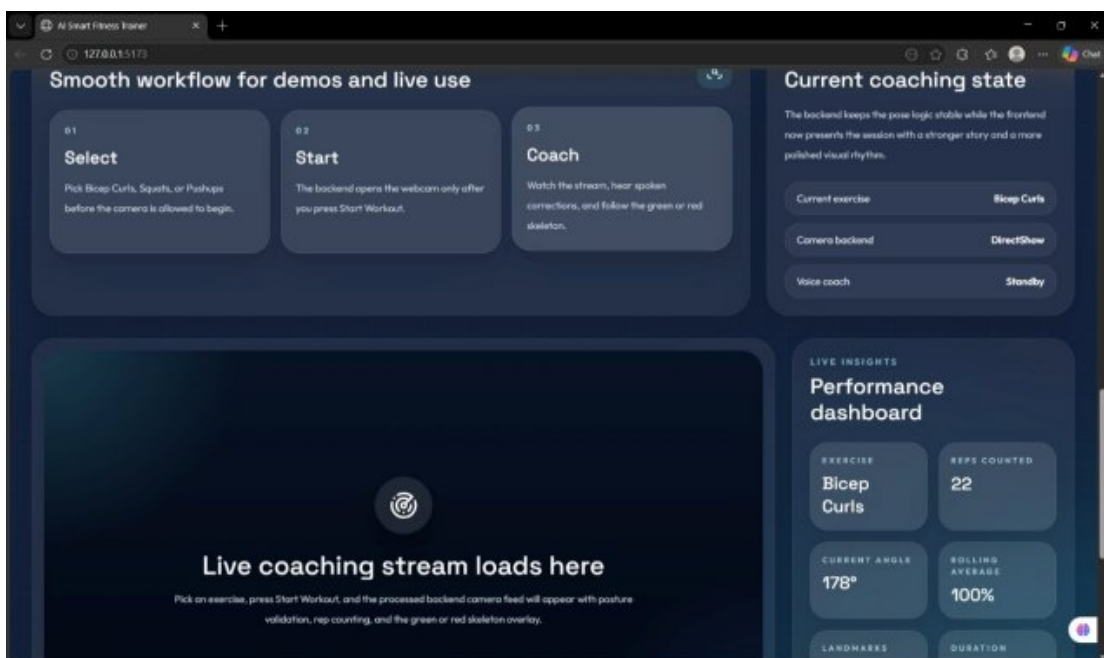


Fig. no. 4.7.3: Interactive Exercise Selection Interface – Bicep Curls, Squats, and Pushups Cards

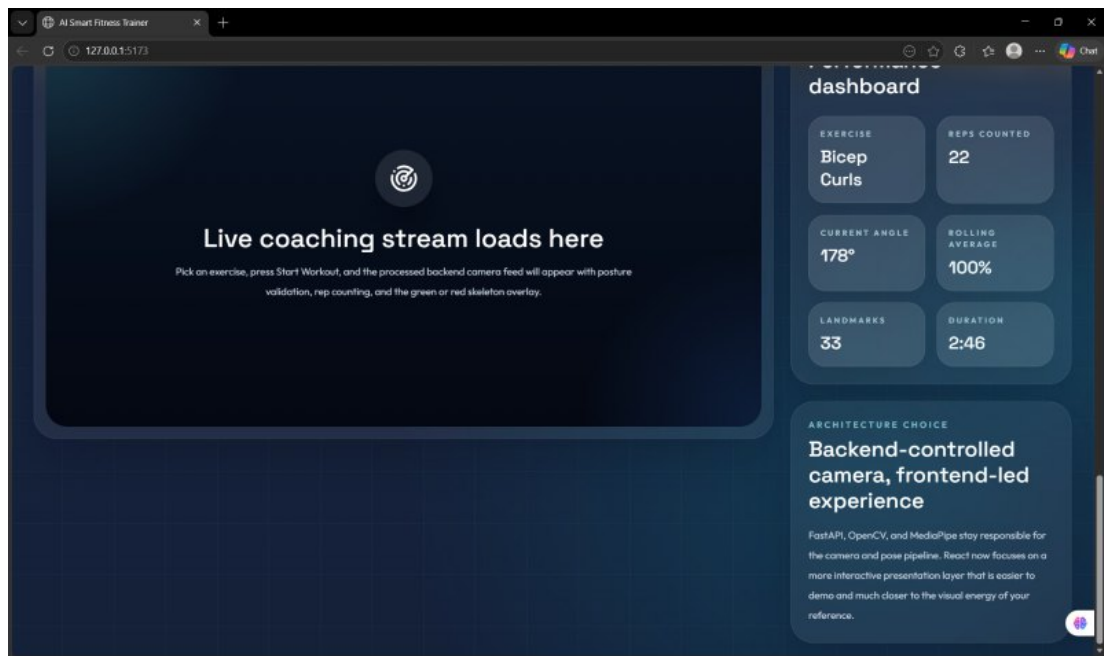


Fig. no. 4.7.4: System Workflow Overview – Exercise Selection, Workout Start, and Real-Time Coaching Process

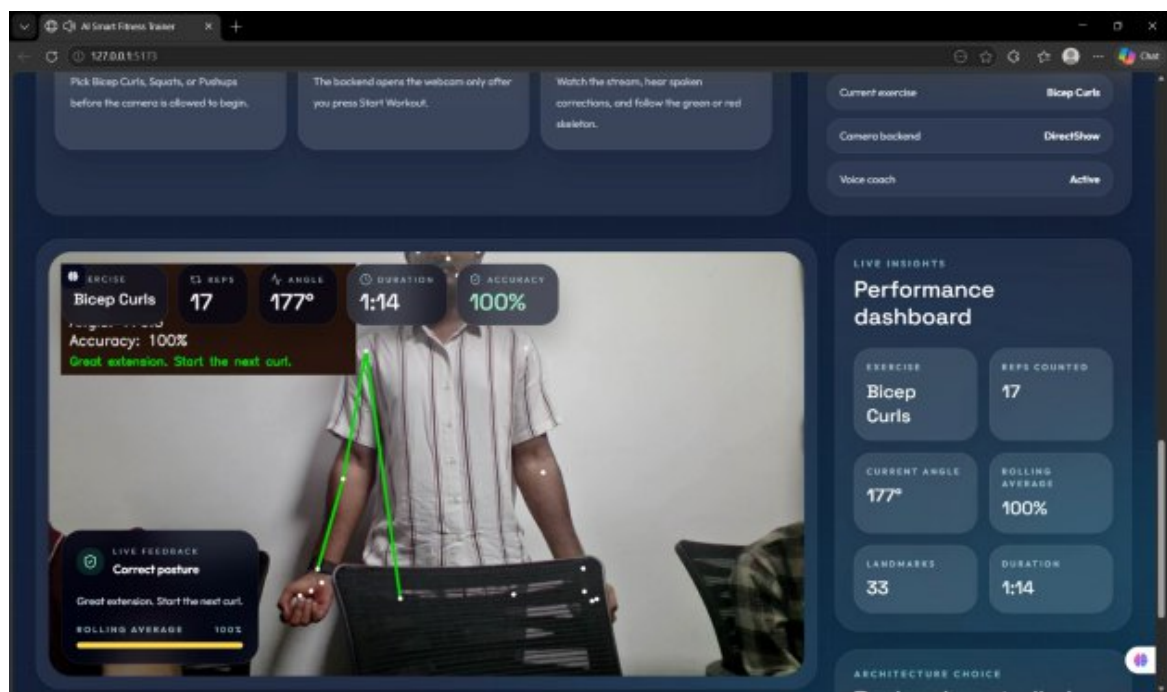


Fig. no. 4.7.5: AI Fitness Trainer – Perfect Posture Detection with 100% Accuracy and Skeleton Alignment Visualization

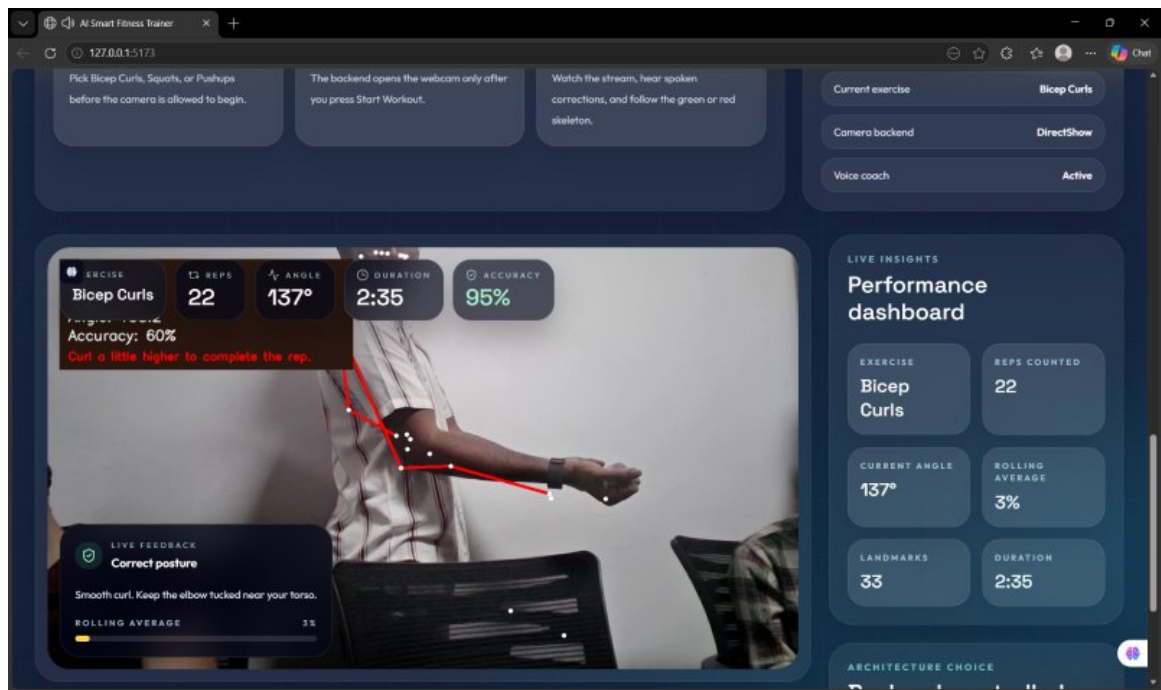


Fig. no. 4.7.6: AI Smart Fitness Trainer – Live Bicep Curl Tracking with Rep Count, Angle Detection, and Real-Time Form Correction

5. CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

The AI Smart Fitness Trainer successfully demonstrates how computer vision can be used to build a practical and interactive workout coaching system. By combining MediaPipe Pose, OpenCV, FastAPI, and a React-based frontend, the project provides a complete workflow for selecting exercises, starting sessions, tracking movement, counting repetitions, validating form, and visualizing results through a live video stream.

The implemented exercise logic for bicep curls, squats, and pushups shows that a rule-based approach can deliver meaningful posture feedback without requiring a custom training pipeline. Rolling accuracy scoring, skeleton color changes, and spoken messages help make the feedback easier to understand while the session is in progress. The backend-controlled camera design also improves operational reliability during live demos and local execution.

Overall, the project is cost-effective, modular, and extensible. It proves that a standard webcam and modern open-source software stack are sufficient to create a smart fitness assistant that supports safer workouts and richer user engagement. The system also provides a solid foundation for future experimentation in personalized exercise analytics.

5.2 Future Scope

The project can be extended by adding support for more exercises such as lunges, planks, shoulder press, and jumping movements. Each exercise can reuse the existing pose pipeline while introducing new angle thresholds, body alignment rules, and rep logic.

A future version can also include user profiles, workout history, and session summaries so that individuals can review performance trends over time. Storing exercise statistics in a database would make it possible to compare sessions and generate personalized improvement insights.

Another important enhancement is the integration of more advanced machine learning models for posture classification, activity recognition, and automatic form scoring. Hybrid systems could combine the current transparent rule-based logic with learned models to improve robustness under different camera angles and body types.

The frontend can be enhanced further with mobile responsiveness, workout plans, audio customization, and richer visual analytics. Multi-camera support, cloud deployment, and shared trainer dashboards could also make the application useful in remote coaching scenarios.

In the future, the application may also support on-device optimization, wearable integration, and real-time error prediction before posture deviations become severe.

These improvements would make the system more intelligent, adaptive, and suitable for



real-world fitness assistance.

REFERENCES

- [1] Z. Cao, T. Simon, S.-E. Wei, and Y. Sheikh, "Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields," CVPR, 2017.
- [2] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv:1906.08172, 2019.
- [3] I. Grishchenko et al., "BlazePose GHUM Holistic: Real-time 3D Human Landmarks and Pose Estimation," arXiv:2206.11678, 2022.
- [4] Google for Developers, "Pose detection," ML Kit documentation, 2024.
- [5] Google for Developers, "Pose classification options," ML Kit documentation, 2024.
- [6] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.
- [7] OpenCV Team, "VideoCapture Class Reference," OpenCV documentation, 2024.
- [8] S. Ramirez, "FastAPI Documentation," FastAPI, 2024.
- [9] Pydantic Team, "Models and BaseModel," Pydantic documentation, 2024.
- [10] React Team, "React Documentation," 2024.
- [11] Vite Team, "Getting Started," Vite documentation, 2024.
- [12] Tailwind Labs, "Styling with Utility Classes," Tailwind CSS documentation, 2024.
- [13] MDN Web Docs, "Web Speech API," Mozilla, 2025.
- [14] NumPy Developers, "Array Objects and ndarray," NumPy documentation, 2024.
- [15] MDN Web Docs, "SpeechSynthesis," Mozilla, 2025.

APPENDIX

main.py

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import StreamingResponse
from pydantic import BaseModel
from threading import Event, Lock, Thread

from exercise_logic import create_initial_state
from pose_module import PoseProcessor
from utils import encode_frame, now_iso, open_camera_capture

class StartSessionRequest(BaseModel):
    exercise: str

class WorkoutSessionManager:
    def __init__(self):
        self.lock = Lock()
        self.worker_thread = None
        self.stop_event = None
        self.latest_frame = None
        self.status = self._idle_status()

    def _idle_status(self):
        return {
            "exercise": None, "counter": 0, "stage": "down",
            "angle": 0.0, "accuracy": 0.0, "averageAccuracy": 0.0,
            "feedback": "Choose an exercise before starting.",
            "isCorrectForm": False, "landmarks": [], "isRunning": False,
            "duration": 0, "error": None, "updatedAt": now_iso(),
        }

    def start(self, exercise: str):
        self.stop()
        initial = create_initial_state(exercise)
        with self.lock:
            self.status = {**initial.to_dict(), "exercise": exercise,
                           "isRunning": True, "updatedAt": now_iso()}
            self.stop_event = Event()
            self.worker_thread = Thread(
                target=self._camera_loop, args=(exercise, self.stop_event), daemon=True
            )
            self.worker_thread.start()

    def stop(self):
        with self.lock:
            if self.stop_event:
                self.stop_event.set()
            worker = self.worker_thread
            if worker and worker.is_alive():
                worker.join(timeout=2)
            with self.lock:
                self.worker_thread = None
                self.stop_event = None
                self.latest_frame = None
                self.status = {**self.status, "isRunning": False, "updatedAt": now_iso()}

    def _camera_loop(self, exercise: str, stop_event: Event):
        camera, backend = open_camera_capture(0)
        processor = PoseProcessor(exercise)
        try:
            while not stop_event.is_set():
                ok, frame = camera.read()
                if not ok:
                    continue
                processed, payload = processor.process_frame(frame)
                encoded = encode_frame(processed)
                if encoded:
                    with self.lock:
                        self.latest_frame = encoded
```

```

        self.status = {"**payload": exercise,
                        "isRunning": True, "cameraBackend": backend,
                        "updatedAt": now_iso()}
    finally:
        processor.close()
        camera.release()

    def get_status(self):
        with self.lock:
            return dict(self.status)

    def frame_generator(self):
        while self.status.get("isRunning"):
            with self.lock:
                frame = self.latest_frame
            if frame:
                yield b"--frame\r\nContent-Type: image/jpeg\r\n\r\n" + frame + b"\r\n"

session_manager = WorkoutSessionManager()
app = FastAPI(title="AI Smart Fitness Trainer API")
app.add_middleware(CORSMiddleware, allow_origins=["http://localhost:5173"],
                  allow_methods=["*"], allow_headers=["*"])

@app.post("/session/start")
def start_session(request: StartSessionRequest):
    session_manager.start(request.exercise)
    return session_manager.get_status()

@app.post("/session/stop")
def stop_session():
    session_manager.stop()
    return session_manager.get_status()

@app.get("/session/status")
def session_status():
    return session_manager.get_status()

@app.get("/video_feed")
def video_feed():
    return StreamingResponse(session_manager.frame_generator(),
                            media_type="multipart/x-mixed-replace; boundary=frame")

```

pose_module.py

```

import cv2
import mediapipe as mp
from collections import deque

from exercise_logic import analyze_exercise, create_initial_state, resolve_tracking_side
from utils import serialize_landmarks

class PoseProcessor:
    def __init__(self, exercise_name: str):
        self.exercise_name = exercise_name
        self.state = create_initial_state(exercise_name)
        self.accuracy_history = deque(maxlen=30)
        self.pose = mp.solutions.pose.Pose(
            static_image_mode=False,
            model_complexity=1,
            smooth_landmarks=True,
            min_detection_confidence=0.5,
            min_tracking_confidence=0.5,
        )

    def process_frame(self, frame):
        frame = cv2.flip(frame, 1)
        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
        results = self.pose.process(rgb)
        landmarks_payload = []

        if results.pose_landmarks:
            landmarks = results.pose_landmarks.landmark

```

```

side = resolve_tracking_side(landmarks)
self.state = analyze_exercise(self.exercise_name, landmarks, self.state, side)
self.accuracy_history.append(self.state.accuracy)
self._draw_pose(frame, landmarks, side, self.state.is_correct_form)
landmarks_payload = serialize_landmarks(landmarks)
else:
    self.state.accuracy = 0.0
    self.state.feedback = "Step into the frame so your full body is visible."

average = round(sum(self.accuracy_history) / len(self.accuracy_history), 1) if self.accuracy_history else 0.0
return frame, {**self.state.to_dict(), "averageAccuracy": average,
               "landmarks": landmarks_payload}

def _draw_pose(self, frame, landmarks, side, is_correct_form):
    color = (0, 220, 0) if is_correct_form else (0, 0, 255)
    # Draw the tracked side skeleton and feedback HUD on the camera frame.

def close(self):
    self.pose.close()

```

exercise_logic.py

```

import math
import numpy as np
from dataclasses import dataclass

EXERCISES = ("bicep-curl", "squat", "pushup")
MOVEMENT_EPSILON = 3

@dataclass
class WorkoutState:
    counter: int
    stage: str
    angle: float
    accuracy: float
    feedback: str
    is_correct_form: bool

    def to_dict(self):
        return {
            "counter": self.counter, "stage": self.stage,
            "angle": round(self.angle, 1), "accuracy": round(self.accuracy, 1),
            "feedback": self.feedback, "isCorrectForm": self.is_correct_form,
        }

    def create_initial_state(exercise: str):
        return WorkoutState(0, "down" if exercise == "bicep-curl" else "up",
                             180.0, 0.0, "Press start to begin tracking.", False)

    def calculate_angle(a, b, c):
        a, b, c = np.array([a.x, a.y]), np.array([b.x, b.y]), np.array([c.x, c.y])
        radians = math.atan2(c[1] - b[1], c[0] - b[0]) - math.atan2(a[1] - b[1], a[0] - b[0])
        angle = abs(math.degrees(radians))
        return 360 - angle if angle > 180 else angle

    def update_bicep_curl(points, state):
        angle = calculate_angle(points["shoulder"], points["elbow"], points["wrist"])
        counter, stage = state.counter, state.stage
        elbow_drift = abs(points["shoulder"].x - points["elbow"].x) > 0.12

        if angle > 160:
            stage = "down"
        if angle < 50 and state.stage == "down" and not elbow_drift:
            stage = "up"
            counter += 1

        accuracy = 45 if elbow_drift else (95 if angle < 50 or angle > 160 else 80)
        feedback = "Keep your elbow closer." if elbow_drift else "Smooth curl."
        return WorkoutState(counter, stage, angle, accuracy, feedback, accuracy >= 70)

    def update_squat(points, state):
        angle = calculate_angle(points["hip"], points["knee"], points["ankle"])

```

```

back_lean = math.degrees(math.atan2(abs(points["shoulder"].x - points["hip"].x),
    abs(points["shoulder"].y - points["hip"].y)))
counter, stage = state.counter, state.stage
if angle < 90:
    stage = "down"
if angle > 160 and state.stage == "down" and back_lean <= 35:
    stage = "up"
    counter += 1
accuracy = 55 if back_lean > 35 else (95 if angle < 90 or angle > 160 else 82)
return WorkoutState(counter, stage, angle, accuracy, "Keep chest lifted.", accuracy >= 70)

def update_pushup(points, state):
    angle = calculate_angle(points["shoulder"], points["elbow"], points["wrist"])
    body_line = calculate_angle(points["shoulder"], points["hip"], points["ankle"])
    counter, stage = state.counter, state.stage
    if angle < 90:
        stage = "down"
    if angle > 160 and state.stage == "down" and body_line > 160:
        stage = "up"
        counter += 1
    accuracy = 52 if body_line <= 160 else (95 if angle < 90 or angle > 160 else 84)
    return WorkoutState(counter, stage, angle, accuracy, "Keep a straight line.", accuracy >= 70)

def analyze_exercise(exercise, landmarks, previous_state, tracked_side):
    points = get_side_points(landmarks, tracked_side)
    if exercise == "bicep-curl":
        return update_bicep_curl(points, previous_state)
    if exercise == "squat":
        return update_squat(points, previous_state)
    if exercise == "pushup":
        return update_pushup(points, previous_state)
    return previous_state

```

useWorkoutSession.ts

```

import { useCallback, useEffect, useState } from "react";
import { API_BASE_URL, DEFAULT_STATUS, normalizeStatus } from "../utils/workout";

const STATUS_POLL_INTERVAL = 400;

export const useWorkoutSession = () => {
    const [status, setStatus] = useState(DEFAULT_STATUS);
    const [isLoading, setIsLoading] = useState(false);
    const [error, setError] = useState<string | null>(null);
    const [streamToken, setStreamToken] = useState(Date.now());

    const fetchStatus = useCallback(async () => {
        const response = await fetch(`${API_BASE_URL}/session/status`);
        const data = normalizeStatus(await response.json());
        setStatus(data);
        setError(data.error);
        return data;
    }, []);

    const startSession = useCallback(async (exercise: string | null) => {
        if (!exercise) {
            setError("Select an exercise before starting the camera.");
            return false;
        }
        setIsLoading(true);
        try {
            const response = await fetch(`${API_BASE_URL}/session/start`, {
                method: "POST",
                headers: { "Content-Type": "application/json" },
                body: JSON.stringify({ exercise }),
            });
            const data = normalizeStatus(await response.json());
            setStatus(data);
            setStreamToken(Date.now());
            return true;
        } finally {
            setIsLoading(false);
        }
    }, []);

```

```

    }
  }, []);

  const stopSession = useCallback(async () =>{
    const response = await fetch(`${API_BASE_URL}/session/stop`, { method: "POST" });
    setStatus(normalizeStatus(await response.json()));
  }, []);

  useEffect(() =>{ void fetchStatus(); }, [fetchStatus]);

  useEffect(() =>{
    if (!status.isRunning) return;
    const id = window.setInterval(() =>void fetchStatus(), STATUS_POLL_INTERVAL);
    return () =>window.clearInterval(id);
  }, [fetchStatus, status.isRunning]);

  return {
    status, error, isLoading,
    streamUrl: status.isRunning ? `${API_BASE_URL}/video_feed?stream=${streamToken}` : null,
    startSession, stopSession,
  };
};

```

Index.tsx

```

import { useState } from "react";
import { useWorkoutSession } from "../hooks/useWorkoutSession";
import { useVoiceCoach } from "../hooks/useVoiceCoach";
import { EXERCISE_OPTIONS } from "../utils/workout";

export const IndexPage = () =>{
  const [selectedExercise, setSelectedExercise] = useState<string | null>(null);
  const { status, error, isLoading, streamUrl, startSession, stopSession } =
    useWorkoutSession();
  useVoiceCoach(status);

  return (
    <main>
      <section>
        <h1>AI Smart Fitness Trainer</h1>
        <p>{status.feedback}</p>

        {EXERCISE_OPTIONS.map((exercise) => (
          <button key={exercise.id} onClick={() => setSelectedExercise(exercise.id)}>
            {exercise.title}
          </button>
        ))}

        <button disabled={!selectedExercise || status.isRunning || isLoading}
          onClick={() => void startSession(selectedExercise)}>
          Start Workout
        </button>
        <button disabled={!status.isRunning} onClick={() => void stopSession()}>
          Stop Workout
        </button>
        {error && <p>{error}</p>}
      </section>

      <section>
        {streamUrl ? (
          <img src={streamUrl} alt="Processed workout stream" />
        ) : (
          <p>Select an exercise and start the camera.</p>
        )}
        <p>Reps: {status.counter}</p>
        <p>Angle: {Math.round(status.angle)} deg</p>
        <p>Accuracy: {Math.round(status.accuracy)}%</p>
        <p>Average: {Math.round(status.averageAccuracy)}%</p>
      </section>
    </main>
  );
};

```

useVoiceCoach.ts

```
import { useEffect, useRef } from "react";

export const useVoiceCoach = (status) => {
  const lastCounterRef = useRef(0);
  const lastMessageRef = useRef({ text: "", at: 0 });

  useEffect(() => {
    if (!("speechSynthesis" in window)) return;
    if (!status.isRunning) {
      window.speechSynthesis.cancel();
      lastCounterRef.current = status.counter;
      return;
    }

    const repChanged = status.counter > lastCounterRef.current;
    const correctionNeeded = !status.isCorrectForm && status.accuracy < 70;
    const message = repChanged ? `${status.counter} reps`
      : correctionNeeded ? status.feedback : null;

    lastCounterRef.current = status.counter;
    if (!message || (message === lastMessageRef.current.text &&
      Date.now() - lastMessageRef.current.at < 3000)) return;

    window.speechSynthesis.cancel();
    window.speechSynthesis.speak(new SpeechSynthesisUtterance(message));
    lastMessageRef.current = { text: message, at: Date.now() };
  }, [status]);
};
```